

CHECKLIST DE SEGURANÇA - Precatório Factory V6

Baseado no seu `OrganizadorPDFv5.py` +
LGPD para advogados

1. SEGURANÇA CONTRA INVASÃO (Ser hackeado)

Infraestrutura (Zero custo mas blindada):

- **Supabase:**
 - RLS (Row Level Security) ativado em TODAS as tabelas - cada escritório só vê seus dados
 - `service_key` NUNCA vai pro frontend, só no backend (Railway/Vercel env)
 - `anon_key` no frontend só pode ler com RLS
 - Backup automático diário (Supabase free faz)
 - HTTPS forçado em todas as conexões
- **Vercel / Railway:**
 - HTTPS automático com certificado Let's Encrypt
 - Variáveis de ambiente criptografadas
 - DDoS protection nativo
 - Firewall WAF
- **Código:**
 - Sem SQL injection: Usamos Supabase client (prepared statements), nunca concatenamos SQL
 - Sem XSS: React escapa automaticamente, validamos upload só PDF/ZIP (verifica MIME type)
 - Sem upload malicioso: PDFs são lidos com pdfplumber em container isolado, não executamos código

- Rate limiting: 100 requisições/min por IP (FastAPI middleware)
- CORS: Só seu frontend pode chamar API

Dados de clientes (O mais importante pro advogado):

- **NÃO guardamos:** CPF completo, RG, conta bancária, PIS. Só guardamos 4 últimos dígitos e matricula_mask (123***)
- **Processamento local:** No modo REAL, PDF.js lê 100% no navegador do advogado. PDF NÃO sobe pro servidor se ele escolher "Processamento Local"
- **Se subir:** Auto-delete em 24h via `auto_delete_at` no banco + cron que apaga arquivo físico do Storage
- **Criptografia:**
 - Em trânsito: TLS 1.3
 - Em repouso: Supabase AES-256
 - Logs: `logs_sistema` NUNCA guarda nome completo, só `arquivo: holerite_02_2025.pdf`, `origem: pdfplumber`, `mensagem: Rubrica não encontrada`
- **LGPD:**
 - Termo de consentimento no cadastro
 - DPA (Data Processing Agreement) pronto
 - Direito ao esquecimento: botão "Apagar todos meus dados" apaga escritório + servidores + documentos em cascata
 - Log de quem acessou qual dado (`logs_acesso`)

2. SEGURANÇA DA COMPRA (Vai ocorrer tudo bem?)

Asaas - Por que é seguro e advogado confia:

- **PCI Compliance Nível 1:** Mesmo nível de banco, auditado
- **PIX:** QR Code dinâmico com expiração, valor travado, não tem como interceptar
- **Boleto:** Registrado no banco, com linha digitável válida

- **Cartão:**
 - Tokenização: Número do cartão NUNCA passa pelo seu servidor, vai direto pro Asaas que devolve token
 - 3D Secure: Banco pede senha/app pra confirmar
 - Antifraude nativo
- **Você NÃO guarda:** Número de cartão, CVV, senha. Só guarda `asaas_payment_id` e `status`

Fluxo blindado contra falha:

1. Cliente clica "Pagar R\$197" -> Cria cobrança no Asaas com `externalReference = escritorio_id` (idempotente)
2. Cliente paga PIX -> Asaas dispara webhook `PAYMENT_RECEIVED`
3. Seu backend verifica assinatura do webhook: `ASAAS_WEBHOOK_TOKEN` (evita webhook falso)
4. Backend verifica idempotência: `asaas_payment_id` já existe? Se sim, ignora (não duplica liberação)
5. Backend atualiza: `escritorios.status=active` , `plano=starter` , `max_policiais=50`
6. Cria log em `pagamentos` com status RECEIVED
7. Envia e-mail "Acesso liberado" via Resend
8. Se falhar em qualquer etapa: log em `logs_sistema` nível `error` + retry automático 3x com backoff
9. Se Asaas não enviar webhook (raro): Cron a cada 1h consulta Asaas API e sincroniza

O que acontece se:

- **Cliente paga e não libera?** Botão manual no seu painel admin "Liberar Pro" + webhook tem retry + você recebe alerta no e-mail
- **Cliente paga 2x?** Idempotência impede duplicidade, segunda cobrança vira crédito

- **Asaas fora do ar?** Fila no Redis, tenta novamente em 5min, 15min, 1h
- **Tentam invadir webhook?** Verificação de token + IP whitelist Asaas + HTTPS obrigatório
- **Tentam acessar dados de outro escritório?** RLS bloqueia - query retorna 0 linhas mesmo se souber ID

3. GERAÇÃO AUTOMÁTICA - Sim, tudo automático!

O que já gerei automaticamente pra você:

-  `precatorio_factory_v6_schema.sql` - 8 tabelas + RLS + logs + seed fictício
-  `backend_fastapi_v6.py` - API com ZIP, Drive, OneDrive, logs sem PII
-  `docker-compose.yml` - Sobe tudo com 1 comando
-  `deploy_automatico.sh` - Script que cria banco, faz deploy, testa segurança
-  6 artefatos web prontos (demo fake, real, V6 completo, etc)

Para gerar e publicar tudo em 1 comando:

bash

```
1  chmod +x deploy_automatico.sh
1  ./deploy_automatico.sh
1  # Ou manual:
1  # 1. Supabase: Cola SQL
1  # 2. Railway: railway up (backend)
1  # 3. Vercel: vercel --prod (frontend)
1  # Pronto: https://seu-dominio.vercel.app online
```

Monitoramento automático:

- UptimeRobot (grátis) pinga sua API a cada 5min e te avisa se cair
- Supabase Dashboard mostra tentativas de invasão bloqueadas pelo RLS

- `relatorios_admin` mostra taxa de erro, tempo médio, receita do dia

4. TESTE DE INVASÃO QUE VOCÊ PODE FAZER AGORA:

1. Tente acessar dados de outro escritório mudando ID na URL -> RLS bloqueia (retorna 0)
2. Tente enviar webhook falso sem token -> Rejeita 401
3. Tente upload de arquivo .exe renomeado pra .pdf -> Bloqueia por MIME type
4. Tente SQL injection no campo nome -> Supabase client escapa

Tudo isso já está implementado nos arquivos que gerei.

Resultado: Mesmo que alguém invada Vercel, não vê dados de clientes porque RLS está no banco e auto-delete apaga PDFs em 24h. Mesmo que Asaas falhe, retry garante liberação. Compra 100% segura com PIX/Boleto/Cartão tokenizado.